

Efficiency of AES Implementations

Carsten Baum & Tyge Tiessen

1 Background

The Advanced Encryption Standard (AES), is one of the most used (if not the most used) current cryptographic algorithm. It has been designed from the ground up to be efficiently implementable in software. Nonetheless, due to its widespread use it has become common for CPU to have dedicated hardware instructions (AES-NI) that allow for an even faster implementation of AES.

In this project, you will implement four versions of AES and determine their respective performance profiles.

2 Tasks

For this project, you need to work in a systems language such as C, C++, or Rust, to get meaningful results.

1. Read and understand the description of AES-128 in “The Design of Rijndael” that you can find with DTU FindIt. Implement a naive version of AES and test it using test vectors that you can also find in that book. Describe the structure and steps of AES, you can skip the key schedule though.
2. In a first attempt at optimizing your implementation, try to determine apparent inefficiencies of the naive implementations and address these.
3. Find an explanation of the optimization technique called “T-tables” for AES. Implement a third version of AES-128 using T-tables.
4. Determine how to utilize AES-NI instructions for an implementation of AES. Implement a last version of AES using AES-NI.
5. Create a benchmark of your four implementations and compare them with each other. Also benchmark an implementation of AES from a

reputable library and compare that to your implementations.

Note: Remember to compile with optimizations enabled.

Note: it is sufficient to only implement and benchmark encryption. Also make sure that you are benchmarking only the encryption and not the key-schedule.

3 Deliverables

Your hand-in should consist of the following two deliverables:

1. A report addressing the tasks listed above.
2. A .zip-file containing the code which you wrote, a README file as well as the test cases.

4 General Information (same for each project)

Both reports and implementations are part of the assessment and therefore graded. The report must be self-contained and assume that the reader has no familiarity with the project tasks being required for this specific project. You thus need to ensure to introduce and describe the concepts used in the project with the required degree of detail and accuracy. You can expect though that the reader has knowledge similar to the course requirements of this course.

Your report must be **at most 12 pages long**, excluding references and project contributions.

Individual contributions. As we need to be able to give grades on an individual basis, it is a requirement that you indicate individual contributions in both report and code. For the report, you can add a short section with a table denoting which student has worked on which parts of the report (and possibly the project in general). Add this section to the end of your report, it will not be counted toward the page limit.

For the code, we recommend to use Git to track individual contributions. If you use Git, remember to submit your code as a Git repository.

Assessment. We will assess the report and code based on rate of completion of the tasks, correctness and appropriateness of the proposed methods/solutions, quality of writing and solution, and adherence to the project requirements.

All project tasks except bonus tasks *must* be completed to receive the highest grade. To achieve a high grade make sure that you cover the tasks in sufficient clarity and detail, and reflect on any issues and possible solutions that you encounter.

Incorrect solutions, inaccurate (or very superficial) reports as well as non-functioning or undocumented code (or code with unintelligible documentation) will lower the grade.

Suggested structure. We recommend that your report has an introduction, followed by necessary background and definitions, and then description of your project and project outcome covering the individual tasks, possibly concluding with a short discussion. Please also do not forget to add references to your report to provide your sources and justify non-trivial claims that you make.

Code submission. If the deliverables contain a programming assignment, then your code must compile and run on a machine running Ubuntu 24.04.2 LTS. Please also strive to adhere to standard practices in software engineering and structure your code in a readable way. Do not forget to properly document your code, so that readers can understand your software architecture and decisions for optimizations.

Please also deliver a README file that has the following three sections:

Compilation and Installation: What are the prerequisites (installed libraries and versions etc.) to compile and run your code, which commands should be used to compile the code, how is the code installed?

Running the test cases: Describe which commands should be used to re-run your test cases to establish that your code is correct. How should the outputs of your tests be interpreted by a reader?

Running benchmarks: If a task requires to benchmark the performance of an implementation, then describe which commands should be used to re-run your benchmarks. How should the outputs of the benchmark scripts be interpreted by a reader?

Please test if the instructions in the README file work in the described environment before handing in.

AI Usage. While we encourage the use of artificial intelligence tools to improve your learning experience, the use of such tools to write your report or code must be clearly indicated. If you fail to do this, the corresponding part will be considered plagiarism. For more information on referencing when using AI, see <https://www.bibliotek.dtu.dk/en/publishing/reference-management/kunstig-intelligens>.